

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Environment Setup

Create a new React application:

Open your terminal and run the following command to create a new React application using Create React App:

```
npx create-react-app react-routing-forms
cd react-routing-forms
```

This command will set up a basic React project structure for you.

Install React Router:

Navigate into your newly created project directory and install react-router-dom, which provides the necessary components for routing:

```
npm install react-router-dom
# or
yarn add react-router-dom
```

Project Structure:

Organize your project directory as follows:

```
react-routing-forms/
├── src/
│   ├── components/
│   │   ├── Home.js
│   │   ├── About.js
│   │   ├── Contact.js
│   │   └── Form.js
│   ├── App.js
│   ├── index.js
│   ├── App.css
│   └── ...
├── public/
├── package.json
└── ...
```

Create the components folder inside the src folder. Within the components folder, create Home.js, About.js, Contact.js, and Form.js.

Step-by-Step Instructions

Step 1: Create Basic Components

Create Home.js:

Inside the components directory, create a file named Home.js and add the following code:

```
import React from 'react';
function Home() {
  return (
```

Home

Welcome to the Home page!

```
    );
  }
  export default Home;
```

This component renders a simple heading and paragraph to represent the home page.

Create About.js:

Create a file named About.js in the same directory and add this code:

```
import React from 'react';
function About() {
  return (
```

About

Learn more about us here.

```
    );
  }
  export default About;
```

This component represents the about page.

Create Contact.js:

Create Contact.js with the following content:

```
import React from 'react';
function Contact() {
  return (
```

Contact

Contact us for any inquiries.

```
    );
  }
  export default Contact;
```

This component represents the contact page.

Create Form.js:

Create Form.js with initial form structure:

```
import React from 'react';
function Form() {
  return (
```

Form

```
    {/ Form fields will go here /}

    );
  }
  export default Form;
```

This will serve as the base for the form component.

Step 2: Implement Routing in App.js

Import necessary modules:

Open App.js and modify it as follows:

```
import React from 'react';
import { BrowserRouter as Router, Route, Routes, Link } from 'react-router-dom';
import Home from './components/Home';
import About from './components/About';
import Contact from './components/Contact';
import Form from './components/Form';
import './App.css';
function App() {
  return (
    <div>
      <h1>React Router</h1>
      <div>
        <Link to="/home">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
        <Link to="/form">Form</Link>
      </div>
      <Routes>
        <Route path="/home" element=<Home /> />
        <Route path="/about" element=<About /> />
        <Route path="/contact" element=<Contact /> />
        <Route path="/form" element=<Form /> />
      </Routes>
    </div>
  )
}
```

```
    );  
  }  
  export default App;
```

This code imports the required components from `react-router-dom` and the components you created earlier. It sets up a Router to enable navigation, Link components for navigation links, and Routes and Route components to map URLs to components.

Add basic styling:

Add some basic styling to `App.css` to make the navigation look better:

```
nav ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
  overflow: hidden;  
  background-color: #333;  
}  
nav li {  
  float: left;  
}  
  
nav li a {  
  display: block;  
  color: white;  
  text-align: center;  
  padding: 14px 16px;  
  text-decoration: none;  
}  
  
nav li a:hover {  
  background-color: #111;  
}
```

This CSS will style the navigation bar at the top of the application.

Step 3: Implement Form Handling in `Form.js`

Add state variables:

Modify `Form.js` to include state variables for form fields:

```
import React, { useState } from 'react';
```

```
function Form() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [message, setMessage] = useState('');

  return (
```

Form

```

    Name:
      type="text"
      id="name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />

    Email:
      type="email"
      id="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />

    Message:
      id="message"
      value={message}
      onChange={(e) => setMessage(e.target.value)}
    />

    Submit

  );
}
export default Form;
```

This introduces useState hooks for the name, email, and message fields. The onChange handlers update the state whenever the input fields change.

Handle form submission:

Add a handleSubmit function to handle the form submission:

```
import React, { useState } from 'react';
function Form() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [message, setMessage] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Form submitted', { name, email, message });
    // You can add logic here to send the form data to a server
    // For example, using fetch or axios
  };

  return (
```

Form

Name:

```
      type="text"
      id="name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
```

Email:

```
      type="email"
      id="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
```

Message:

```
      id="message"
      value={message}
      onChange={(e) => setMessage(e.target.value)}
    />
```

Submit

```
    );  
  }  
  export default Form;
```

The `handleSubmit` function prevents the default form submission behavior and logs the form data to the console. In a real application, you would replace the `console.log` with code to send the data to a server.

Step 4: Implement Form Validation

Add validation logic:

Enhance `Form.js` to include form validation:

```
import React, { useState } from 'react';  
function Form() {  
  const [name, setName] = useState('');  
  const [email, setEmail] = useState('');  
  const [message, setMessage] = useState('');  
  const [errors, setErrors] = useState({});  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    const validationErrors = {};  
  
    if (!name) {  
      validationErrors.name = 'Name is required';  
    }  
  
    if (!email) {  
      validationErrors.email = 'Email is required';  
    } else if (!/^^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/\\.test(email)) {  
      validationErrors.email = 'Invalid email format';  
    }  
  
    if (!message) {  
      validationErrors.message = 'Message is required';  
    }  
  
    setErrors(validationErrors);  
  
    if (Object.keys(validationErrors).length === 0) {  
      console.log('Form submitted', { name, email, message });  
      // You can add logic here to send the form data to a server  
      // For example, using fetch or axios  
    }  
  }  
};
```



```
return (
```

Form

Name:

```
type="text"
```

```
id="name"
```

```
value={name}
```

```
onChange={(e) => setName(e.target.value)}
```

```
{errors.name &&
```

```
{errors.name}
```

```
}
```

Email:

```
type="email"
```

```
id="email"
```

```
value={email}
```

```
onChange={(e) => setEmail(e.target.value)}
```

```
{errors.email &&
```

```
{errors.email}
```

```
}
```

Message:

```
id="message"
```

```
value={message}
```

```
onChange={(e) => setMessage(e.target.value)}
```

```
{errors.message &&
```

```
{errors.message}
```

```
}
```

Submit

```
);  
}  
export default Form;
```

This code adds validation checks for each field. If a field is invalid, an error message is displayed below the input. The form submission is only processed if there are no validation errors.

Step 5: Run the Application

Start the development server:

In your terminal, run:

```
npm start  
# or  
yarn start
```

This will start the React development server, and you can view your application in your browser at <http://localhost:3000>.

Test the application:

Navigate through the different routes using the navigation links. Fill out the form and submit it. Observe the console output and the validation messages.

Validation & Expected Results

Routing: Clicking on the navigation links should navigate you to the corresponding pages (Home, About, Contact, Form).

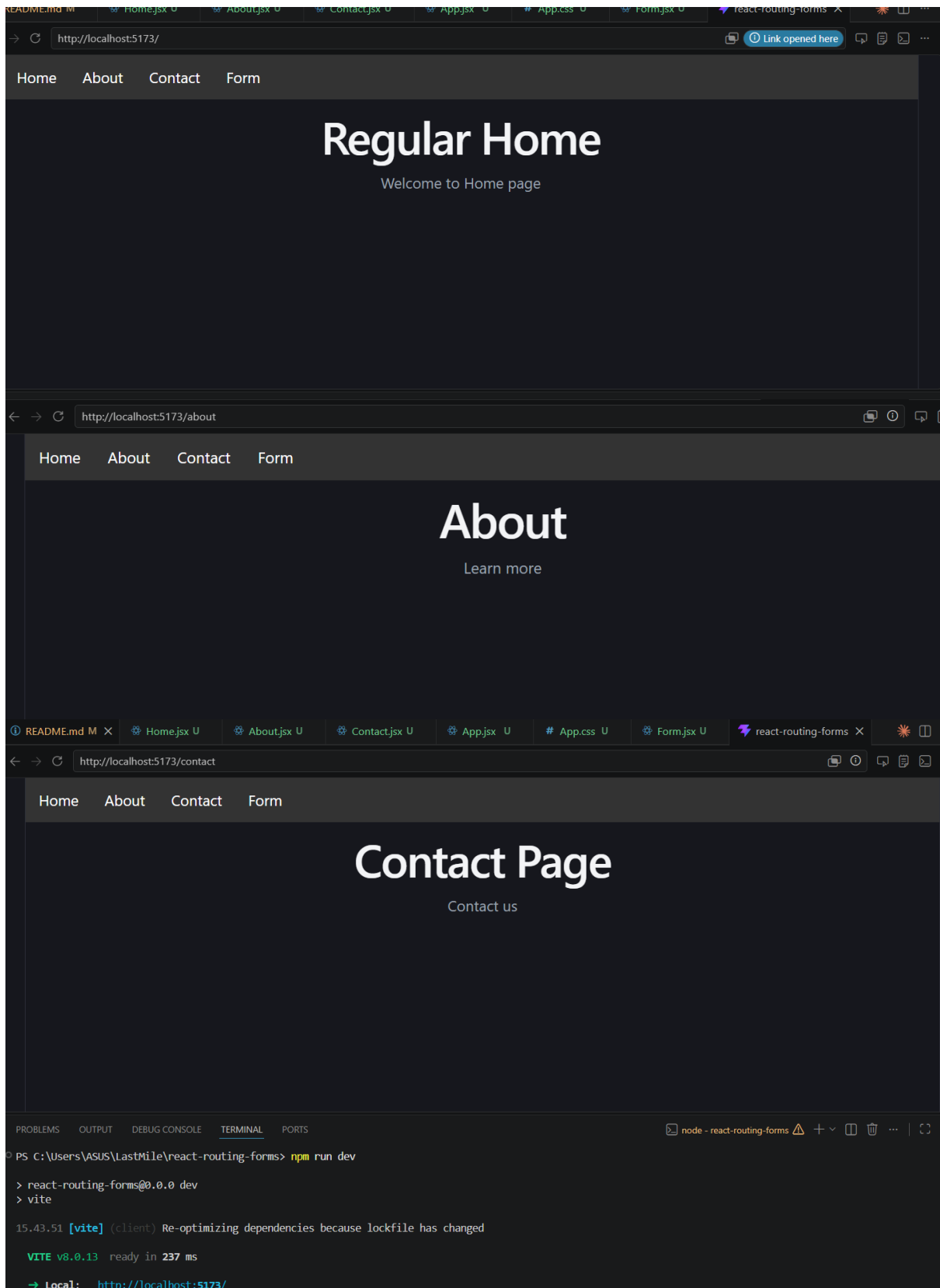
Form Input: Typing in the form fields should update the component's state, which is reflected in the input values.

Form Submission: Submitting the form without filling in the required fields should display error messages. Submitting the form with valid data should log the form data to the console.

Expected console output upon successful form submission:

```
Form submitted {name: 'Your Name', email: 'your.email@example.com', message: 'Your message'}
```

Answer:



HomeAboutContactForm

Form

Name:

Email:

Message:

Submit

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

node - react-routing-forms

PS C:\Users\ASUS\LastMile\react-routing-forms> npm run dev
> react-routing-forms@0.0.0 dev
> vite

15.43.51 [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.0.13 ready in 237 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

HomeAboutContactForm

Form

Name:

asda

Email:

asda

asua

Submit

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

node - react-routing-forms

PS C:\Users\ASUS\LastMile\react-routing-forms> npm run dev
> react-routing-forms@0.0.0 dev
> vite

15.43.51 [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.0.13 ready in 237 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

Please include an '@' in the email address. 'asda' is missing an '@'.

The screenshot shows a web browser window with a dark theme. The browser's address bar displays "chrome://devtools/". The page has a navigation bar with links: "Home", "About", "Contact", and "Form". The main content area is titled "Form" and contains three input fields labeled "Name:", "Email:", and "Message:". The "Name:" and "Email:" fields are text inputs, while the "Message:" field is a larger text area. Below these fields is a "Submit" button. The browser's developer tools are open on the right side, showing the "Console" tab. The console displays the message "Form submitted:" followed by an object containing the following properties: "email": "asid@gmail.com", "message": "asida", and "name": "asida". The object is also shown in the "Object" tab of the developer tools.